High Performance Computing

PGAS and UPC Exercise

Environment

This exercise uses concepts and syntax from **Unified Parallel C (UPC)**.

UPC compiler commands depend on the implementation installed on the system. Use the UPC environment provided for the course or laboratory.

A typical UPC source file has the extension:

`program.upc`

and contains:

```
1 #include <upc.h>
```

Before starting the programming parts, verify that you can compile and run a simple UPC program in the environment provided by the instructor.

Objectives

The purpose of this exercise is to understand the basic ideas of the Partitioned Global Address Space (PGAS) programming model and to practice them using UPC. You will examine shared and private memory, determine data affinity, distinguish between local and non-local accesses, distribute loop iterations among threads, use shared arrays, and apply synchronization when threads depend on data produced by other threads.

Part 1: Understanding the PGAS Model

Consider a program running with four threads:

$$T_0, \qquad T_1, \qquad T_2, \qquad T_3.$$

Each thread has private memory, while part of the program data belongs to a partitioned global address space.

Answer the following questions.

1. What does PGAS stand for?
2. What is meant by a **global address space**?
3. Why is the global address space described as **partitioned**?
4. What is meant by the affinity of a shared data item?
5. What is a local memory access?
6. What is a non-local memory access?

7. Can a UPC thread access shared data that has affinity to another thread?
8. What important difference exists between non-local access in MPI and non-local access in a PGAS model?

Part 2: Shared and Private Variables

Assume:

$$\texttt{THREADS} = 4.$$

Consider:

```
1 shared int x;
2 shared int y[THREADS];
3 int z;
```

Answer the following.

1. How many copies of `x` exist?
2. Which thread has affinity to `x`?
3. How many elements does `y` contain?
4. Which thread has affinity to each element of `y`?
5. How many copies of `z` exist?
6. Is `z` shared between the four threads?

Complete the table.

Variable	Shared / Private	Affinity
`x`		
`y[0]`		
`y[1]`		
`y[2]`		
`y[3]`		
`z`		

Part 3: Blocking and Affinity

UPC allows shared arrays to be distributed among threads using blocks.
Consider:

```
1 shared [4] int A[32];
```

and assume:

$$\texttt{THREADS} = 4.$$

The affinity of an element can be determined using:

$$\text{thread}(i) = \left\lfloor \frac{i}{\text{block size}} \right\rfloor \bmod \texttt{THREADS}.$$

For this example:

block size = 4.

Complete the table.

Array Elements	Thread with Affinity
$A[0]$–$A[3]$	
$A[4]$–$A[7]$	
$A[8]$–$A[11]$	
$A[12]$–$A[15]$	
$A[16]$–$A[19]$	
$A[20]$–$A[23]$	
$A[24]$–$A[27]$	
$A[28]$–$A[31]$	

Answer:

1. Which thread has affinity to $A[2]$?
2. Which thread has affinity to $A[6]$?
3. Which thread has affinity to $A[13]$?
4. Which thread has affinity to $A[22]$?
5. Why might the choice of block size affect program performance?

Part 4: First UPC Program

Consider the following UPC program:

```
1  #include <stdio.h>
2  #include <upc.h>
3
4  int main()
5  {
6      printf("Hello from thread %d of %d\n",
7             MYTHREAD, THREADS);
8
9      return 0;
10 }
```

Run the program using four UPC threads.
Record the output.

Thread	Printed Message
0	
1	
2	
3	

Answer:

1. What does `MYTHREAD` represent?
2. What does `THREADS` represent?

3. Does every UPC thread execute the same program?
4. Is the order of the printed messages guaranteed?
5. In what way is this execution model similar to MPI?

Part 5: Parallel Vector Addition

Consider the vector operation:

$$C_i = A_i + B_i.$$

Use:

$$N = 1000.$$

Create three shared arrays:

```
#define N 1000

shared int A[N];
shared int B[N];
shared int C[N];
```

Initialize the arrays so that:

$$A_i = i$$

and

$$B_i = 2i.$$

Then calculate:

$$C_i = A_i + B_i.$$

Use `upc_forall` to distribute the iterations among UPC threads.
You may use the following structure:

```
#include <stdio.h>
#include <upc.h>

#define N 1000

shared int A[N];
shared int B[N];
shared int C[N];

int main()
{
    int i;

    upc_forall(i = 0; i < N; i++; &A[i])
    {
        A[i] = i;
        B[i] = 2 * i;
    }

    upc_barrier;

```

```
    upc_forall(i = 0; i < N; i++; &C[i])
    {
        // Calculate C[i]
    }

    upc_barrier;

    if (MYTHREAD == 0)
    {
        printf("C[0]   = %d\n", C[0]);
        printf("C[10]  = %d\n", C[10]);
        printf("C[999] = %d\n", C[999]);
    }

    return 0;
}
```

Complete and run the program.
Verify that:

$$C_i = 3i.$$

Questions

1. Can different values of C_i be calculated independently?
2. What is the purpose of `upc_forall`?
3. What does the fourth expression of `upc_forall` specify?
4. Why is distributing work according to data affinity potentially useful?
5. Does every thread calculate all N elements?

Part 6: Observing Data Affinity

Use a smaller shared array:

```
shared int A[16];
```

Assume four threads.
UPC provides:

```
upc_threadof(&A[i])
```

which returns the thread having affinity to the shared element.
Write a program where thread 0 prints the affinity of every element:

```
if (MYTHREAD == 0)
{
    for (int i = 0; i < 16; ++i)
    {
        printf("A[%d] has affinity to thread %d\n",
               i,
               (int)upc_threadof(&A[i]));
    }
}
```

Complete the table from your output.

Element	Thread with Affinity
$A[0]$	
$A[1]$	
$A[2]$	
$A[3]$	
$A[4]$	
$A[5]$	
$A[6]$	
$A[7]$	

Answer:

1. What is the default block size of a UPC shared array?
2. Does the distribution repeat after all threads receive one block?
3. Would accessing an element having affinity to `MYTHREAD` normally be considered local or non-local?

Part 7: Local and Non-Local Access

Assume:

```
1 shared int A[THREADS];
```

and:

$$\texttt{THREADS} = 4.$$

Each element has affinity to the corresponding thread.
Consider thread 0.
Classify the following accesses as primarily **local** or **non-local** from the point of view of thread 0.

Access	Local / Non-local
`A[0]`	
`A[1]`	
`A[2]`	
`A[3]`	

Now consider thread 2.

Access	Local / Non-local
`A[0]`	
`A[1]`	
`A[2]`	
`A[3]`	

Answer:

1. Why can excessive non-local accesses reduce performance?
2. Why should work preferably be assigned to a thread that has affinity to the data being processed?
3. How is this idea related to data locality discussed earlier in the course?

Part 8: Synchronization

Consider:

```
shared int result[THREADS];
```

Every thread writes its own result:

```
result[MYTHREAD] = MYTHREAD * 10;
```

Thread 0 then prints all values.
Complete the following program:

```
#include <stdio.h>
#include <upc.h>

shared int result[THREADS];

int main()
{
    result[MYTHREAD] = MYTHREAD * 10;

    // Add synchronization here

    if (MYTHREAD == 0)
    {
        for (int i = 0; i < THREADS; ++i)
        {
            printf("result[%d] = %d\n",
                   i, result[i]);
        }
    }

    return 0;
}
```

Use:

```
upc_barrier;
```

before thread 0 reads the results.
Run the program with four threads.
The expected values are:

$$0, \quad 10, \quad 20, \quad 30.$$

Questions

1. Why is a barrier required before thread 0 reads all results?
2. What does a thread do when it reaches `upc_barrier` before the other threads?
3. What potential problem could occur if the barrier were removed?
4. Does UPC automatically synchronize threads after every shared-memory operation?

Part 9: Shared Pointers

Consider the following declarations:

```
int *p1;
shared int *p2;
int *shared p3;
shared int *shared p4;
```

Complete the table.

Pointer	Pointer Location	Pointed-to Data
p1		
p2		
p3		
p4		

Use the terms:

- private;
- shared.

Answer:

1. Which pointer type can be used to access arbitrary locations in the shared address space?
2. What is meant by a pointer-to-shared?
3. What locations may an ordinary C pointer safely reference in the UPC memory model?

Part 10: Dynamic Shared Memory

UPC provides different methods for dynamically allocating shared memory.
Consider:

```
upc_global_alloc(...)
```

and:

```
upc_all_alloc(...)
```

Answer the following.

1. Which operation is collective?
2. Which operation is normally expected to be called by one thread?
3. When `upc_all_alloc()` is called, do all threads receive the same shared pointer?
4. What function is used to free dynamically allocated shared memory?

Part 11: Comparing Programming Models

Complete the table.

Model	Multiple Memories	Non-local Access	Explicit Messages
OpenMP			
MPI			
PGAS / UPC			

Then answer:

1. What feature of PGAS makes it look similar to shared-memory programming?
2. What feature of PGAS makes data locality still important?
3. In MPI, how would a process normally obtain data stored by another process?
4. In UPC, how can a thread obtain shared data having affinity to another thread?
5. Why can PGAS be viewed as a model between traditional shared-memory and message-passing programming?

Part 12: Short Discussion

Answer each question briefly.

1. What is data affinity in PGAS?
2. What is the difference between shared and private data in UPC?
3. What is the purpose of a blocked shared array?
4. What is the purpose of `upc_forall`?
5. Why are local accesses generally preferable to non-local accesses?
6. What is the purpose of `upc_barrier`?
7. What is the difference between a blocking barrier and a split-phase barrier?
8. What problem can locks solve when several threads write to the same shared data?
9. What is one major conceptual difference between UPC and MPI?
10. What is one major conceptual difference between UPC and OpenMP?

Submission

Submit:

1. the first UPC thread-identification program;
2. the parallel vector addition program;
3. the data-affinity experiment;
4. the synchronization experiment;
5. completed affinity and memory-layout tables;
6. answers to all questions.

All source files should contain clear comments identifying the corresponding exercise part.